

MANUAL DEL PROGRAMADOR

Actividad 5: Modelado de Datos con Estructuras (STRUCT) y Punteros

Materia: Laboratorio de Programación

Curso: 5° 3°

Lenguaje: C++

Archivo principal: src/main.cpp

Versión: 1.0.0

1. Introducción

Este manual explica el funcionamiento del programa desarrollado para la Actividad 5 de Laboratorio de Programación.

El objetivo principal es aprender a utilizar **estructuras (struct)** y **punteros en C++**, permitiendo organizar diferentes datos dentro de una misma entidad y modificarlos mediante un puntero utilizando el operador flecha (->).

Estos conceptos también pueden servir como base para representar entidades o componentes del Proyecto Integrador Anual.

2. Estructura utilizada

El programa utiliza una estructura llamada EntidadProyecto:

```
struct EntidadProyecto {  
    int id;  
    char nombre[50];  
    float metrica;  
};
```

La estructura agrupa tres datos diferentes:

- **id:** identifica a la entidad y es de tipo entero (int).
- **nombre:** almacena el nombre o descripción y es un arreglo de caracteres (char).
- **metrica:** guarda un valor numérico decimal y es de tipo float.

La ventaja de utilizar un struct es que permite agrupar datos relacionados dentro de una misma entidad, en lugar de tener muchas variables separadas.

3. Creación e inicialización de la entidad

En la función main() se crea una variable llamada miEntidad:

```
EntidadProyecto miEntidad = {0, "Vacio - [Su Nombre y Apellido]", 0.0f};
```

De esta manera, la estructura comienza con valores iniciales antes de que el usuario ingrese los datos.

Los valores iniciales son:

- id = 0
- nombre = "Vacio - [Su Nombre y Apellido]"
- metrica = 0.0

4. Punteros

El programa utiliza un puntero para trabajar con la estructura.

El prototipo de la función es:

```
void cargarDatos(EntidadProyecto* ptr);
```

El * indica que ptr es un puntero a una variable de tipo EntidadProyecto.

Un puntero almacena la dirección de memoria de otra variable. En este caso, permite que la función `cargarDatos()` trabaje directamente sobre `miEntidad`.

5. Pasaje por dirección

Para enviar la dirección de `miEntidad` a la función se utiliza el operador `&`:

```
cargarDatos(&miEntidad);
```

El operador `&` obtiene la dirección de memoria de la variable.

De esta forma, la función recibe un puntero que apunta a la estructura original y puede modificar sus datos directamente, sin crear una copia de toda la estructura.

6. Operador flecha (->)

El operador flecha permite acceder a los miembros de una estructura cuando se trabaja mediante un puntero.

Dentro de `cargarDatos()` se utiliza:

```
ptr->id  
ptr->nombre  
ptr->metrica
```

Por ejemplo:

```
cin >> ptr->id;
```

Esto significa que el valor ingresado por el usuario se guarda directamente en el campo `id` de la estructura a la que apunta `ptr`.

La diferencia es:

```
miEntidad.id
```

cuando se tiene directamente la estructura.

Y:

```
ptr->id
```

cuando se tiene un puntero que apunta a esa estructura.

7. Ingreso del ID

El programa solicita al usuario un número entero:

```
cout << "=> Ingrese el ID de la entidad (entero): ";  
cin >> ptr->id;
```

El dato ingresado se almacena en el campo id utilizando el operador flecha.

8. Limpieza del buffer con cin.ignore()

Después de ingresar el ID se utiliza:

```
cin.ignore();
```

Esto es necesario porque después de utilizar cin >> queda un Enter (\n) en el buffer de entrada.

Si no se limpia, el siguiente getline() podría tomar ese Enter como si fuera una entrada y no permitir que el usuario escriba correctamente el nombre.

Por eso se utiliza:

```
cin.ignore();
```

antes de:

```
cin.getline(ptr->nombre, 50);
```

9. Ingreso del nombre o descripción

Para ingresar el nombre o descripción se utiliza:

```
cin.getline(ptr->nombre, 50);
```

getline() permite ingresar una cadena de texto y el número 50 indica el máximo de caracteres que se pueden almacenar en el arreglo nombre.

El dato también se carga utilizando el operador flecha.

10. Ingreso de la métrica

Finalmente, el programa solicita una métrica de operación:

```
cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";  
cin >> ptr->metrica;
```

El valor ingresado puede contener decimales y se guarda en el campo `metrica`, que es de tipo `float`.

Esta métrica puede representar, por ejemplo, una lectura de un sensor, un consumo, un porcentaje de avance u otro valor numérico relacionado con el proyecto.

11. Verificación de los datos

Una vez finalizado el ingreso, el programa vuelve a `main()` y muestra los datos almacenados:

```
cout << "ID Registrado: " << miEntidad.id << endl;  
cout << "Nombre Registrado: " << miEntidad.nombre << endl;  
cout << "Metrica Guardada: " << miEntidad.metrica << endl;
```

También muestra la dirección de memoria de la estructura:

```
cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
```

Esto permite comprobar que los datos fueron almacenados en la estructura y observar la dirección de memoria correspondiente.

12. Flujo general del programa

El funcionamiento del programa puede resumirse en los siguientes pasos:

1. Se define la estructura `EntidadProyecto`.
2. Se crea e inicializa `miEntidad`.
3. Se obtiene su dirección mediante `&`.
4. Se envía esa dirección a `cargarDatos()`.
5. La función recibe la dirección mediante el puntero `ptr`.
6. Se ingresan los datos utilizando el operador `->`.
7. Se utiliza `cin.ignore()` antes de `getline()`.
8. Se almacenan el ID, nombre y métrica.
9. La función termina y vuelve a `main()`.
10. Se muestran los datos ingresados y la dirección de memoria.

13. Compilación y ejecución

Para compilar el programa desde la terminal de PowerShell de Visual Studio Code se utiliza:

```
g++ src/main.cpp -o src/sistemaproyecto.exe
```

Después, para ejecutar el programa:

```
.\src\sistemaproyecto.exe
```

14. Archivos relacionados

La estructura recomendada para el repositorio es:

```
sistemaproyecto/  
├── .gitignore  
├── LICENSE  
├── README.md  
├── docs/  
│   ├── InformeEEST1_LPR2026_ACT05_G03_v1.0.0.pdf  
│   └── manuales/  
│       ├── manual_programador_v1.0.0.pdf  
│       └── manual_programador_v1.0.0.md  
├── src/  
│   └── main.cpp  
└── capturas/  
    └── ejecucion_struct.png
```

El archivo principal del programa se encuentra en `src/main.cpp`, mientras que el manual forma parte de la documentación ubicada dentro de `docs/manuales/`.